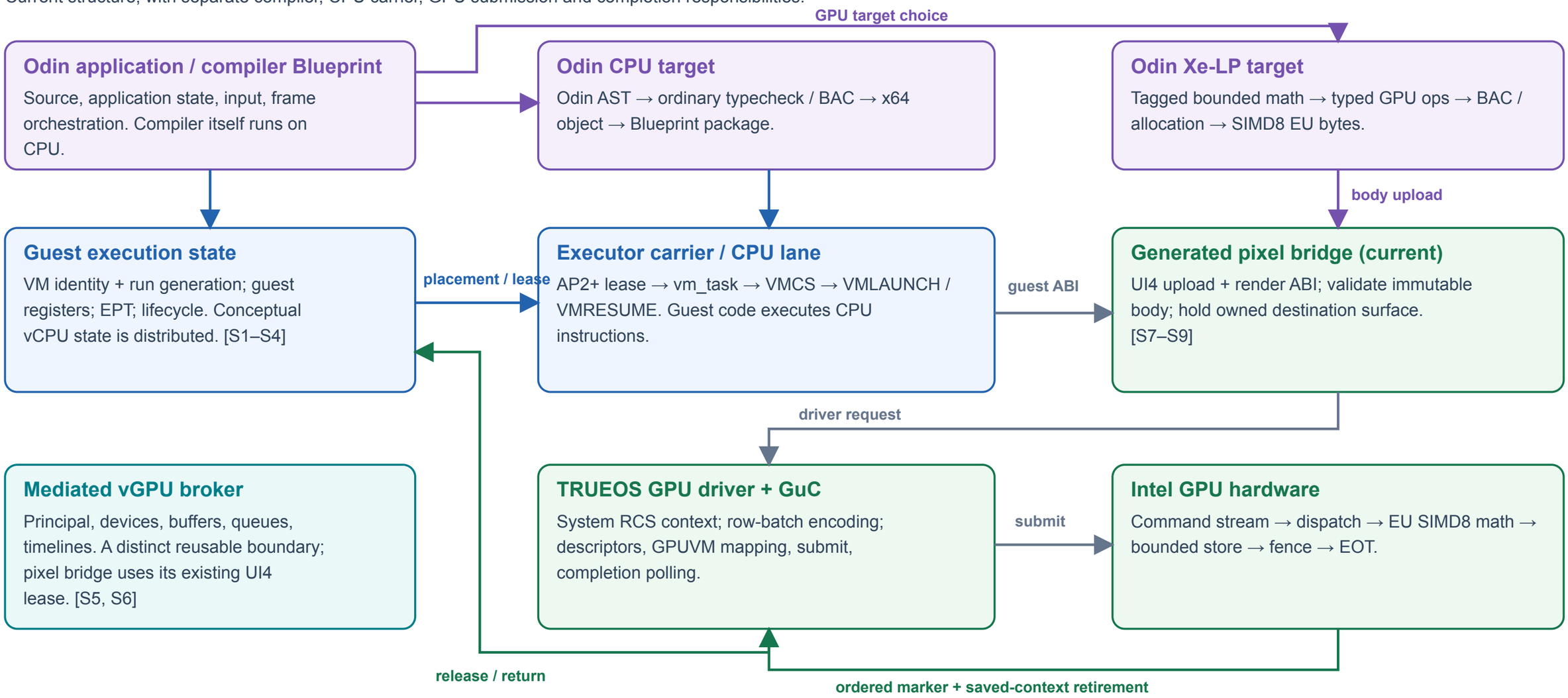
One application; two execution engines

Current structure, with separate compiler, CPU carrier, GPU submission and completion responsibilities.



Coordination can be shared; execution contracts stay explicit

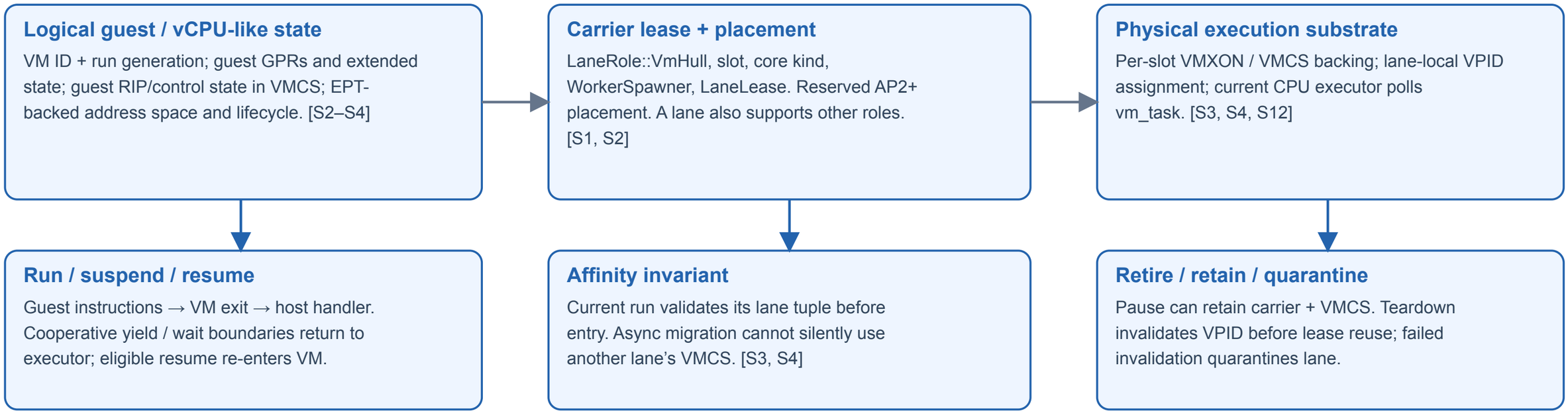
CPU carriers run tasks / guest contexts. GPU EUs execute admitted kernels. A common work graph can connect them; today there is no single demonstrated CPU/GPU submission API.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

Code review: 24 Oct 2026 • no new rig run

Where the vCPU abstraction lives today

A guest execution context, an executor lane, a CPU SIMD element and a GPU SIMD element have different identities.



Terms to keep separate in design discussions

VM / vCPU-like context	Owns execution state and lifecycle; currently spread across VM, VMX and lane modules.
Executor carrier lane	Host scheduling resource / CPU slot lease. Its role can be Hull, worker or blocking work.
CPU SIMD lane	One element in an AVX/SIMD operation; not a VM identity or scheduling lease.
GPU SIMD8 lane	One data element of an EU instruction. The pixel body calculates eight pixels in parallel.
vGPU queue / GPU context	Virtual admission / completion identity versus physical firmware scheduling context.

Interpretation: name the logical CPU execution contract explicitly; a new guest-visible vCPU API remains a design choice.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

Three GPU compiler paths, three different contracts

The handwritten encoder remains valuable. Backend reuse and runtime integration are separate questions.

A · Native pixel math body

Ordinary f32 / bool math; bounded source loops unrolled by frontend.

Shared BAC machinery

Typed GPU ops → BAC/SSA → target lowering → register allocation → SIMD8 arithmetic body; r1/r2 input, r4 output.

Current integration + recorded proof

UI4 bridge + runtime envelope; eight useful pixel lanes; GPU writes surface. ADL-S matching image hashes recorded on 22 Oct 2026. [S7–S10]

B · Native scalar-call kernel

Two f32 arguments → one f32 result. CPU stub embeds .gpu.xelp code.

Shared BAC + kernel tail

Same math compiler, plus result-store / fence / EOT convention. Runtime call is generated into CPU code.

Current ABI gap

Compiler requests trueos_cabi_vgpu_call_f32_v1, marked retired by current loader. Host exposes handle-based ...vgpu_scalar_f32_v1 instead. [S6, S11]

C · Legacy boids command DSL

gpu_add_scaled produces four-word commands; optional fixed-register EU body export.

Separate legacy lowering

Fixed r16..r20 boids arithmetic export; legacy body has no shared scalar-path allocation. Four-word DSL ≠ native EU encoding.

Transport still needs tracing

Optional EU body is a fragment. Legacy CPU object retains its command dispatch route; vFile:compute-boids is not established by this review.

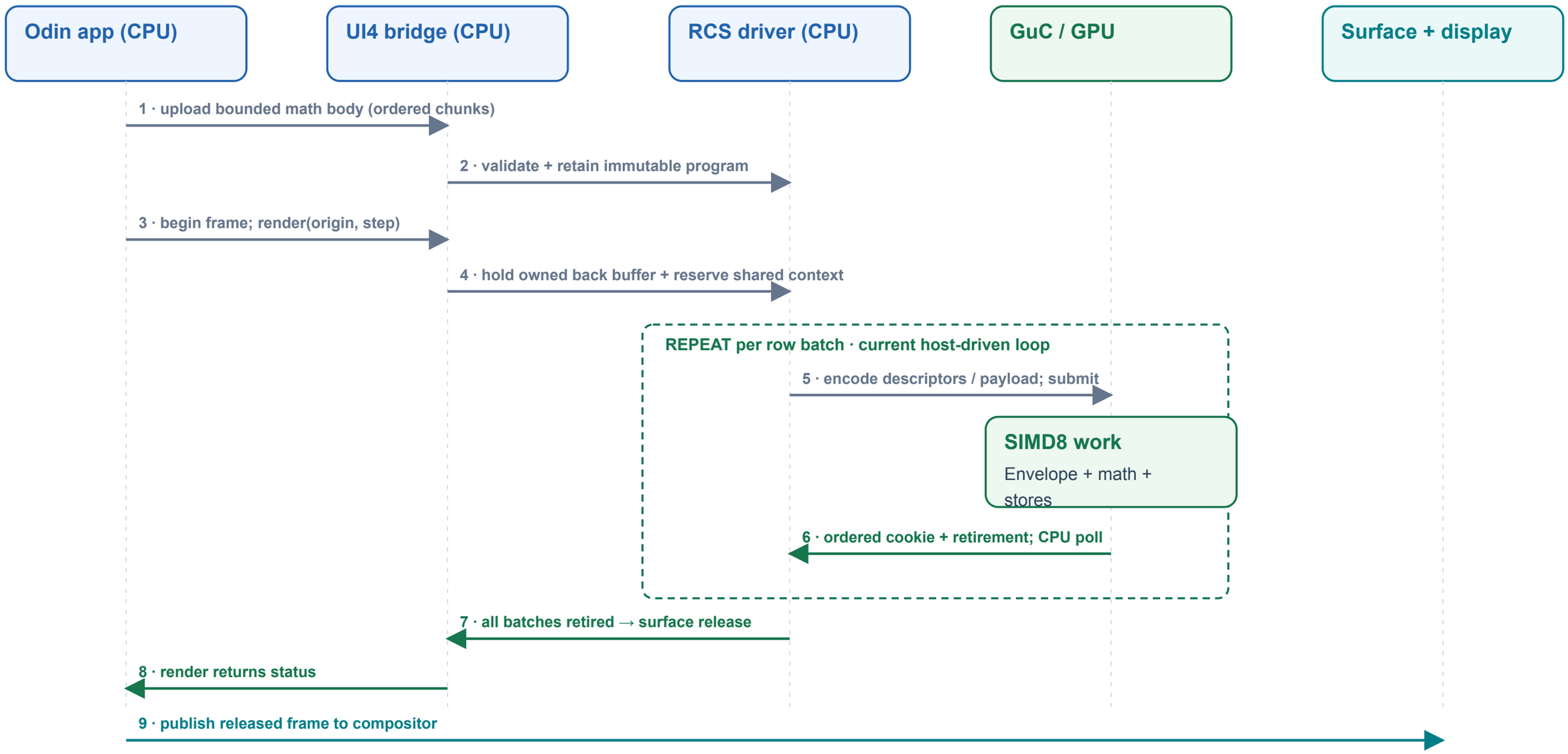
What “scalar source” means here

SIMD8 arithmetic is already emitted. Pixel body inputs vary across all eight elements; scalar-call ABI exposes one active result lane. Wider batching needs invocation IDs, memory access and an explicit ABI. 16-byte uncompacted encoding says how bytes are stored, not how many useful data elements execute.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

A frame today: control crosses the CPU boundary per row batch

Sequence diagram • vertical order is causal; spacing is schematic and represents no measured time.

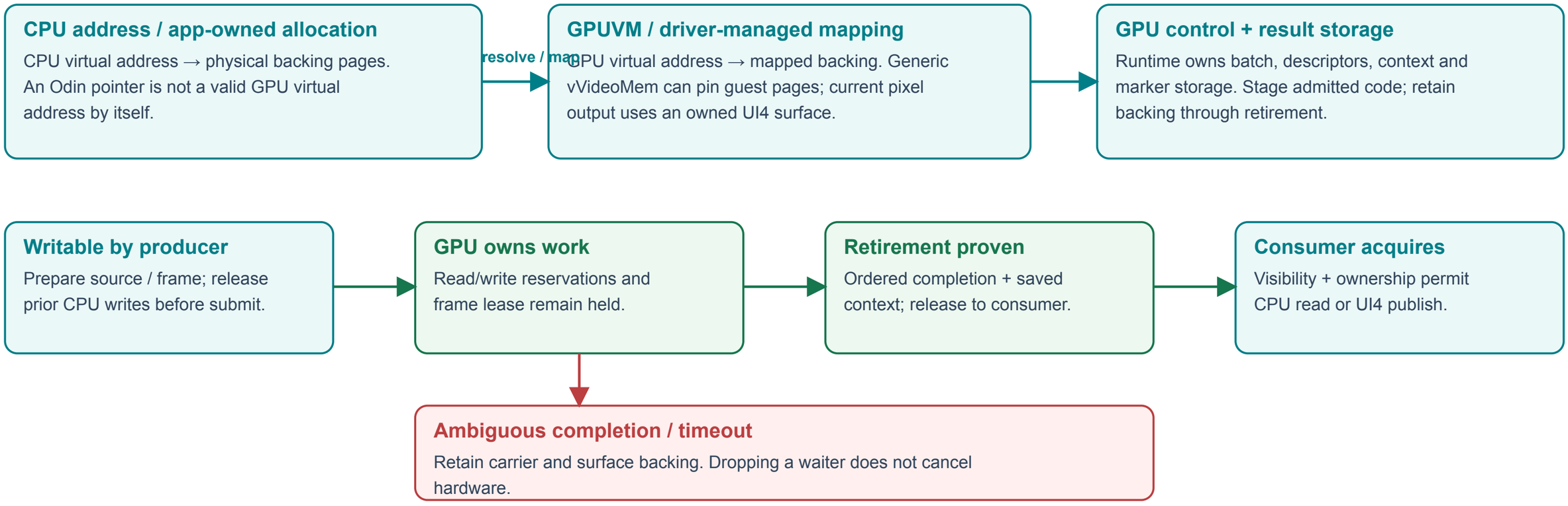


A single application render call currently expands into multiple submits and waits. No CPU callback occurs between math instructions.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

Memory ownership and the four meanings of synchronization

An integrated GPU can access system RAM; address translation, visibility and ownership still need explicit contracts.



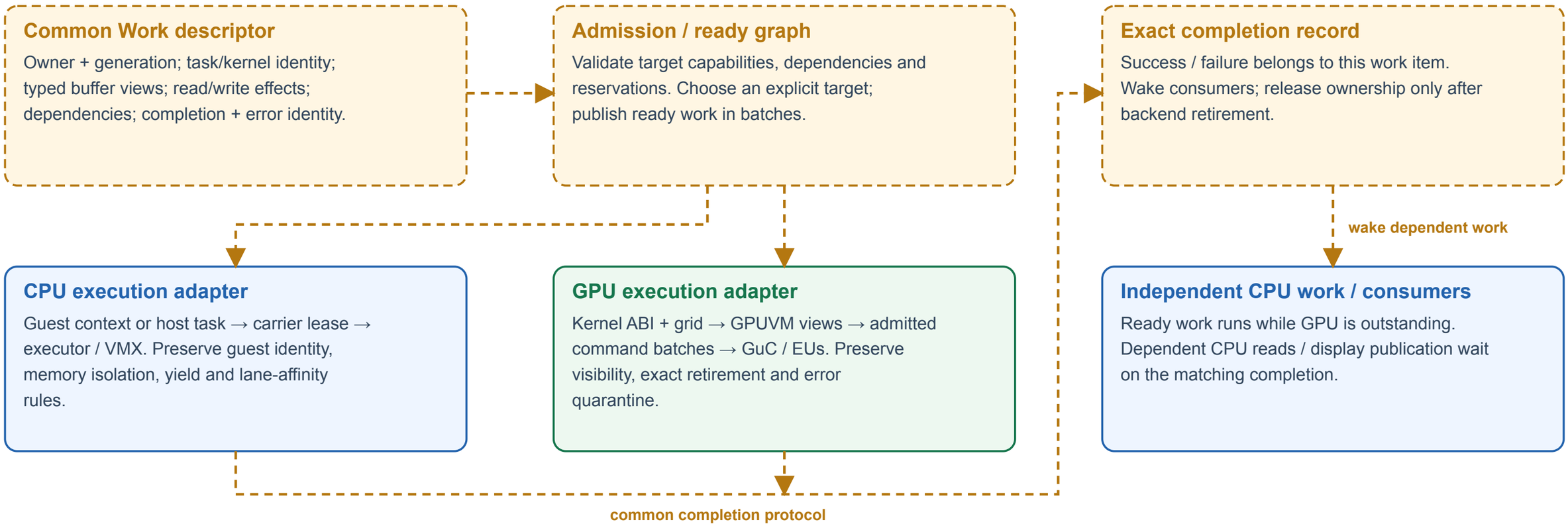
1 · Instruction dependency	Register / ALU / SEND readiness inside an EU thread; conservative @1 in generated math.
2 · Device memory visibility	Data-cache / memory fence orders writes; EU EOT terminates a thread.
3 · Queue and context retirement	Ordered marker plus context proof permits host resource reuse and completion accounting.
4 · Application dependency	A consumer waits for its exact producer; a frame publishes only after its release token exists.

Workgroup barriers are another contract; the bounded straight-line math body does not implement general workgroup synchronization.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

A possible common work model over CPU and GPU backends

DESIGN PROPOSAL • A shared dependency / ownership protocol can coexist with distinct execution engines.



What to optimize and measure

Cold work = compile + admit / prepare + map / visibility + submit + GPU execution + retire / acquire.

Warm work can reuse admitted code. Batch independent elements to amortize submission; split only for real dependencies / limits.

Compare against scalar and vectorized CPU execution on identical data, including end-to-end latency and useful CPU overlap.

SIMD width, independent hardware threads, workgroup count and instruction-level parallelism are separate tuning dimensions.

A device-resident worklist needs its own validated protocol and memory operations; the current math-body subset does not provide it.

CPU / host · Compiler · GPU / driver · Memory / data · Dashed: proposed or gap · Failure / quarantine

How to read the architecture

One coordination model, two execution engines is a useful proposed architecture. Today, CPU guest execution uses executor-backed VMX carriers, and GPU work uses mediated capabilities plus driver / firmware execution. The GPU admission executor is host-side code; it does not run the EU math. Odin's compiler runs on the CPU, even when generating GPU code.

The CPU side already contains vCPU-like state. Its representation is distributed across guest registers, VMCS/EPT, VM identity, run generation and lane assignments. A carrier lane is the host resource that runs this state; a stable logical vCPU identity could be documented or factored above it. A full public multi-vCPU model, migration policy or unified scheduler is not established by this review.

There are also **two GPU ABI layers** to keep distinct: a complete scalar-call kernel and a pixel arithmetic body with a runtime-owned envelope. Their active-lane and result conventions differ. The legacy boids DSL adds a third historical route. These are not interchangeable meanings of “two runtimes.”

> **Current integration gap:** odin-son still emits `trueos_cabi_vgpu_call_f32_v1` with a pointer-based signature. Current TRUEOS rejects that import as retired and exposes the handle-based `trueos_cabi_vgpu_scalar_f32_v1`. This static finding does not invalidate the separate recorded pixel-body execution. No runtime implementation was changed for these diagrams.

What the recorded GPU run establishes

The checked-in 22 October report records ADL-S execution of runtime-compiled Mandelbrot bodies with matching independent image hashes. The 25-iteration body is 14,496 bytes and the 16-iteration body is 9,168 bytes. It establishes this bounded generated-code path. This review did not run the rig, establish a speedup, or prove arbitrary Odin programs on the GPU.

The encoder deliberately emits uncompacted 16-byte instructions. Its SIMD8 arithmetic already operates on vector registers. Pixel input contains eight independent coordinates; the scalar-call ABI exposes one active result lane. A native instruction's byte length alone cannot determine program size limits, useful parallelism, or CPU/GPU performance.

Batching and the “one submit” objective

The current pixel driver loops over row batches, submitting and polling each batch. One application render call therefore does not imply one hardware submit. A future resident worklist could reduce host crossings, but requires device memory operations, bounded descriptors, dependency rules and a termination/completion protocol beyond the current arithmetic-only body.

For performance work, record compilation, admission, mapping / visibility, queueing, submit, execution, retirement and acquisition separately. Compare identical workloads against a vectorized CPU baseline. Long GPU sequences can amortize overhead, while register pressure, divergence, code expansion and independent work supply determine throughput. Matrix instructions are a separate feature to establish and validate; they are not shown as available in this target.

Source map and evidence

Solid boxes describe checked-in components; amber dashed boxes and arrows are proposals or identified gaps. Evidence is a mixture of current static code and an explicitly dated hardware report. The attached author-review PDF was background context, not executable instructions. Local code resolves differences with that older review.

ID	Source	Supports
S1	**CPU carrier roles and leases** src/hv/lane.rs:24	LaneRole, LaneLease, pick_carrier_lane; role and placement are distinct from guest state.
S2	**Hull placement contract** src/hv/guest_work.rs:6	Executor-backed Hull carriers; AP2+ default; VMX backing is indexed per CPU slot.

ID	Source	Supports
S3	**VM task and VM entry loop** src/hv/mod.rs:4090	vm_task carries lane lease; VM entry loop validates assignment and re-enters the guest.
S4	**Guest register state and VPID contract** src/hv/vmx.rs:239	GuestRegisters plus extended state. See also tools/docs/vpid_runtime_validation.md for invalidation / affinity.
S5	**GPU async admission / completion executor** src/gpu/executor.rs:1	KernelSubmission, GpuFence, waiter wakeup, exact retirement. This is not itself a GPU instruction interpreter.
S6	**vGPU scalar broker and C ABI** src/r/io/vgpu_cabi.rs:30	Handle-based trueos_cabi_vgpu_scalar_f32_v1; resolves to broker and native scalar driver.
S7	**UI4 generated compute boundary** src/ui4/blueprint_ui4/native_compute_api.rs:28	Chunk upload, owner/frame checks, feature gating; synchronous render_wire call and release handling.
S8	**Generated pixel dispatch + math envelope** src/intel/gpgpu/operations/generated_pixel.rs:148	Host row-batch submit/poll loop. Envelope at line 400 owns coordinates, packing, store, fence / EOT.
S9	**Direct RCS submission and retirement** src/intel/gpgpu/rcs/commands.rs:892	Poll helpers and saved-context retirement. Also rcs/runtime.rs:316 documents marker + saved-head proof.
S10	**Recorded physical Mandelbrot evidence** docs/experiments/odin-runtime-mandelbrot.md:117	22 Oct 2026 ADL-S checksums for constant, 25- and 16-iteration images; report and JSON, not a new run.
S11	**Retired compiler import** src/hv/blueprint/prebind.rs:85	Old pointer-based trueos_cabi_vgpu_call_f32_v1 is explicitly rejected as retired.
S12	**CPU executor polling** src/runtime.rs:23	Local CPU executor polling; run_ap_forever; supporting VPID contract in tools/docs/vpid_runtime_validation.md.
S13	**Odin native GPU lowering and encoder contracts** ../odin-son/odinc/gpu_native.odin:10	Compiler body / scalar stub. See also typecheck/gpu_native.odin:212, bac/intel_xelp/README.md and encode.odin.

Additional sources: `crates/trueos-v/src/vgpu.rs`, `src/gpu/vgpu.rs:2008`, `src/intel/gpgpu/operations/native_scalar.rs`, `odin-son/odinc/typecheck/gpu.odin`, `odin-son/trueos/mandelbrot.odin`, `tools/docs/vpid_runtime_validation.md`. Attached PDF: *odin-son author change review, 227a6ee excluded through 343588a*, prepared 23 October 2026.